X-HEEP

3rd session

How to integrate easy-to-plug SW for X-HEEP

"a flexible HW/SW framework"

Jose Miranda
jose.miranda@upm.es

CEI UPM Centro de Electrónica Industrial

HEEP-UP Torino 26

POLITÉCNICA

CEI at UPM

CEI is a research center affiliated to the Universidad Polítécnica de Madrid (UPM)

- UPM is the largest technical university in Spain, also the first in terms of employability.
- Number 86 in engineering and technology according to *QS World University Ranking*.
- UPM is also the first Spanish university in the obtention of external resources for R&D&I.
- CEI is the UPM research center specialized on **power electronics** and **digital embedded systems**.

Power Electronics

- Power Converters
- Power Supply Architectures
- Modeling & Simulation
- Smart Grids
- Energy Harvesting

Digital Embedded Systems

- Reconfigurable Embedded Systems
- OpenHW and RISC-V
- Internet of Things
- Machine Learning at the Edge
- Post-Quantum Cryptography

Some Numbers

- 57 Full-time researchers
  - 19 Doctors (Faculty, Contracted researchers)
  - 38 Full-time Students (18 Doctorate, 20 Master)
- 19 Part time, sponsored students (Bachelor)
- 3 Administration & Technicians

Members of :

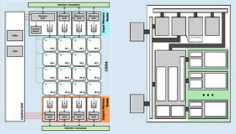
MICROELECTRÓNICA μe-UPM

Digital Embedded Systems at CEI

RISC-V based SoC design and custom accelerators

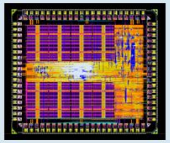
Adaptive RISC-V processors, domain-specific ISA extensions, mixed-criticality, heterogeneous multicores, fault tolerant adaptive systems and digital accelerators for spiking NNs.

Reconfigurable hardware and adaptive computing



Reconfigurable computing infrastructures for FPGAs, CGRAs, compilers and tools for reconfigurable systems, evolutionary computation.

Microelectronic Design for digital systems



ASIC implementation of digital SoCs (TSMC 65 nm), Generative AI for system-level and physical design.

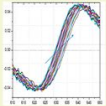
Internet of Things

Networked embedded systems to face challenges related to the era of smart and sustainable cities, comprising the integration of heterogeneous hardware and software technologies.

Cybersecurity for embedded systems

Classical and Post quantum cryptography in IoT, HW accelerators, side channel attacks protection for quantum safe, integration with RISC-V, HW accelerators for QKD key distillation and management

Machine Learning at the Edge



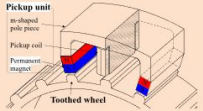
Embedded circuits for near-sensor decision making, based on HW/SW embedded signal processing and machine learning techniques.

Smart Wearables

Combination of multi-parametric sensing with adaptive and context-aware sampling, Edge AI technologies and Affective Computing.

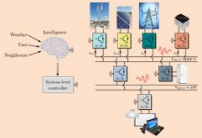
10 Professors
14 full-time PhD students
4 Industrial PhDs.
> 15 Part-time Master students

Energy Harvesting and energy optimization for IoT



Integration of multiple sources, storage, and IoT sensor devices, Collaborative AI-powered Computation offloading strategies.

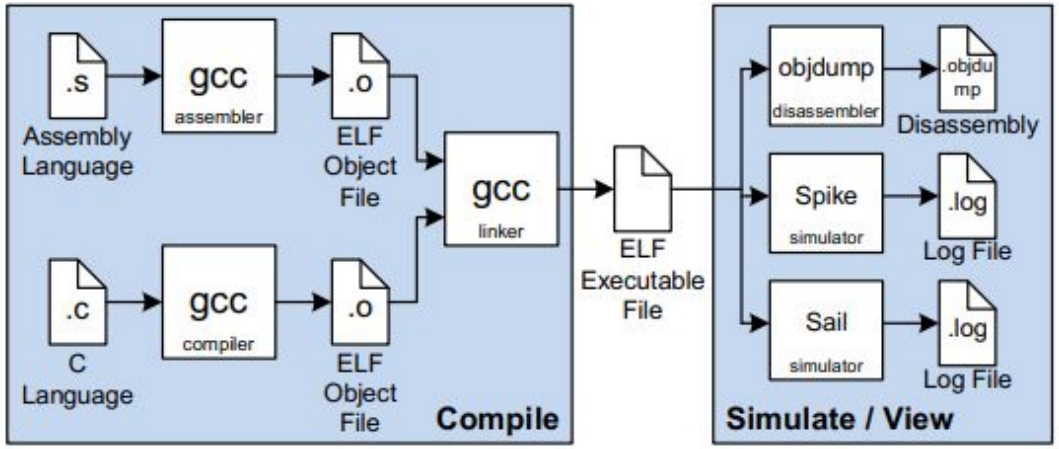
Smart Grids



AC & DC power distribution architectures, blackbox modeling of power converters, system-level control strategies, integration of IoT technology, and stability analysis.

https://github.com/des-cei

General RISC-V Embedded Software Tool Flow

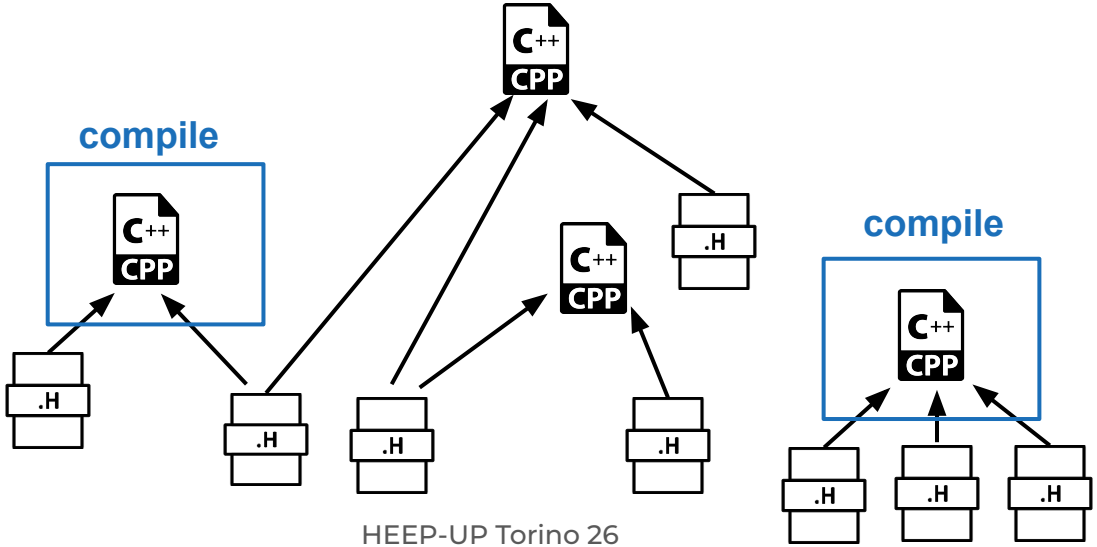


Compile source files into ELF executable

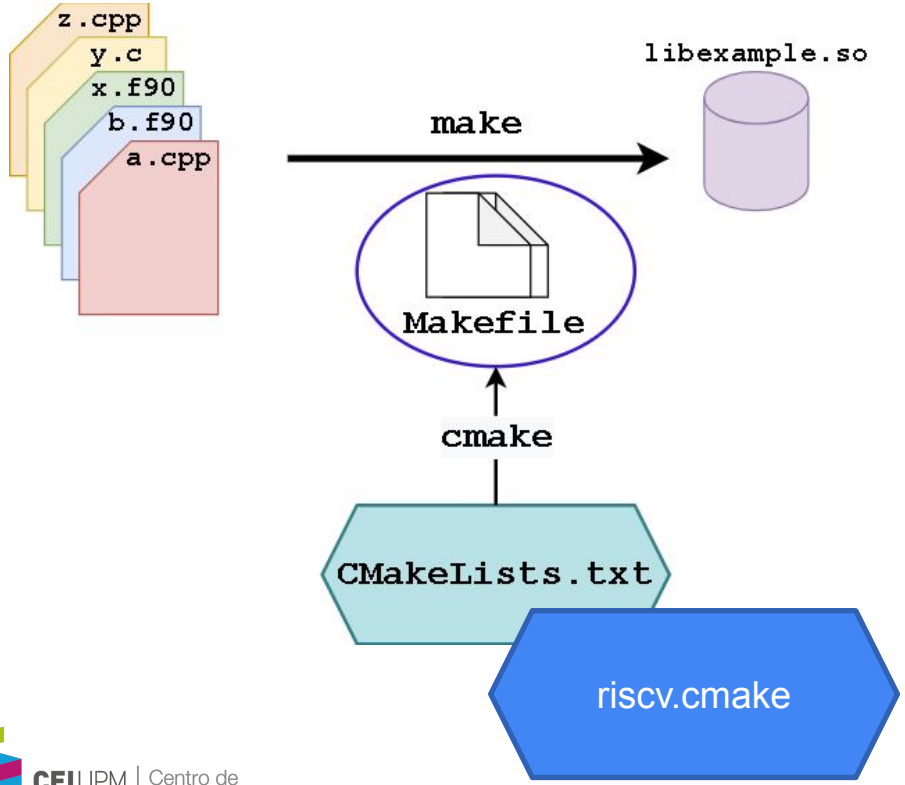
Simulate program running (Spike, Sail) or View compiled program (objdump)

Build Systems motivation

- Why do we need them (Build Systems)?
- Are you able to keep track of ALL the files?
- Are you able to build and maintain a ***Makefile*** with MANY different targets (one per APP)? (from a 177-lines to a 734-lines Makefile)
- Build Systems are able to automatize the compilation and linking process



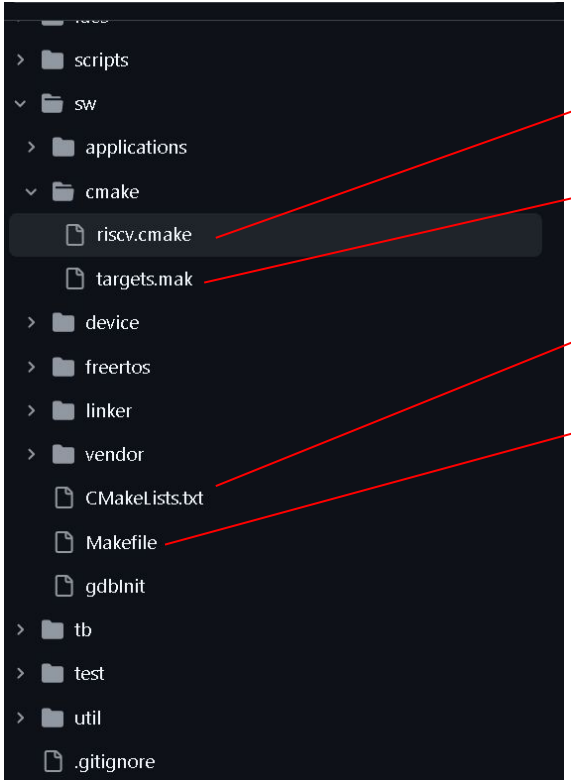
CMake into X-HEEP



- We can encode all the needed (platform/toolchain dependent) PATHs and CMAKE_FLAGS into an auxiliary file that is passed/included into the main CMakeLists.txt engine

https://five-embeddev.com/baremetal/cmake/

CMake into X-HEEP

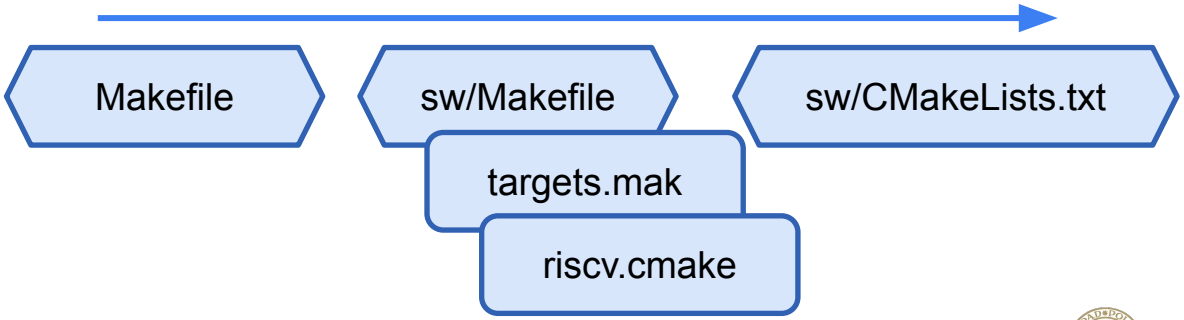


...Defines specific settings for the RISC-V toolchain and CMake compilation options

...Defines build targets, rules, and auxiliary build parameters

...Describes the project structure and what CMake must build

...Orchestrates the build, run, and clean flow



Linker MAGIC

- A linker script contains four things:
  - **Memory layout**: what memory is available where
  - **Section definitions**: what part of a program should go where
  - **Options**: commands to specify architecture, entry point, ...etc. if needed
  - **Symbols**: variables to inject into the program at link time

  - No rules about the total number of Sections
  - Sections used to have 2 addresses: the LMA and the VMA
    - LMA: Load Memory Address (where the data is placed at startup)
    - VMA: Virtual Memory Address (where the data is to be placed/found at running/execution time)

Linker MAGIC + X-HEEP Configurability

make mcu-gen X_HEEP_CFG=configs/general.hjson

make mcu-gen X_HEEP_CFG = configs/python_unsupported.hjson PYTHON_X_HEEP_CFG = configs/general.py

x-heep / configs / general.py ↑ Top

Code Blame 94 lines (83 loc) · 3.39 KB Raw

```python
def config():
    system = XHeep(BusType.onetoM)
    system.set_cpu(cv32e20(rv32e=False, rv32m="RV32MSlow"))

    memory_ss = MemorySS()
    memory_ss.add_ram_banks([32] * 2)
    memory_ss.add_linker_section(LinkerSection.by_size("code", 0, 0x00000E800))
    memory_ss.add_linker_section(LinkerSection("data", 0x00000E800, None))
    system.set_memory_ss(memory_ss)
```

Linker MAGIC + X-HEEP Configurability

make mcu-gen X_HEEP_CFG=configs/general.hjson

make mcu-gen X_HEEP_CFG = configs/python_unsupported.hjson PYTHON_X_HEEP_CFG = configs/general.py

x-heep / configs / python_unsupported.hjson

davideschiavone and ThomasLenges add flash controller (#795) (#854) 6c1e2df · 2 months ago History

Code Blame 89 lines (84 loc) · 2.84 KB Raw

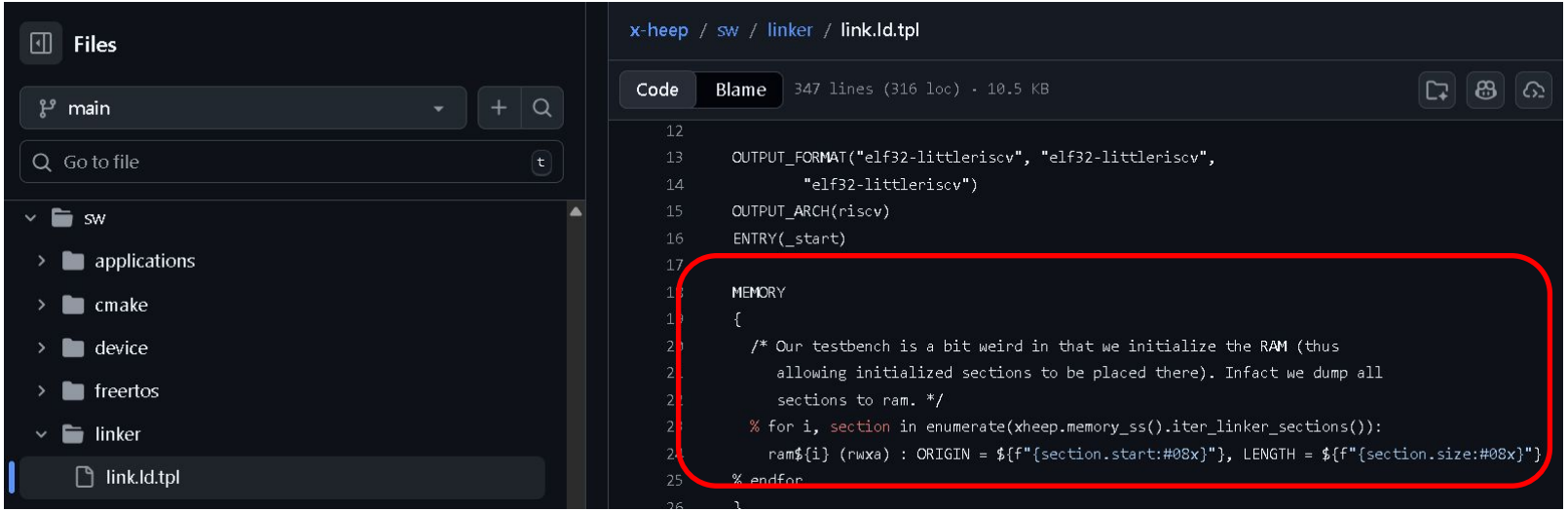
```
// Copyright 2025 EPFL
// Solderpad Hardware License, Version 0.51, see LICENSE for details.
// SPDX-License-Identifier: SHL-0.51
// X-HEEP configurations not yet supported by the Python modelling.
//
// WARNING! To be used together with a Python configuration. This does not
// contain all parameters required to configure X-HEEP.

{
    linker_script: {
        stack_size: 0x800
        heap_size: 0x800
    }

```

Linker MAGIC + X-HEEP Configurability

- We rely on .ld.tpl files to make the linker configurable based on previous parameters...



Linkers and RUNTIME Environments

- Link.ld.tpl:
  - Used for **JTAG-based execution**
  - Application is loaded directly into internal SRAM
  - Suitable for debugging flows
  - No flash boot/execution involved
- Link_flash_load.ld.tpl:
  - Used for **flash boot with RAM execution**
  - Application is stored in flash and copied to internal SRAM at startup
  - Useful when execution speed from SRAM is preferred
  - Requires a startup copy from flash to RAM (i.e. LMA to VMA)
- Link_flash_exec.ld.tpl:
  - Used for **direct execution from flash**
  - Application remains stored and executed in flash
  - Helps reduce SRAM usage, but MUCH slower!!
  - Suitable when XIP-like execution is desired

The start-up Journey!

boot_sel_i	execute_from_flash_i	boot procedure
0	X	JTAG
1	1	SPI Flash Execution
1	0	SPI Flash Loading

- MAIN is a FUNCTION

1. PoR (Power-On-Reset):
   - MCU is first powered up
   - All peripherals and internal logic are reset
2. BootROM execution:
   - Boot code is executed (***x-heep/hw/ip/boot_rom/boot_rom.S***)
   - Mainly it decides what is the device boot source (Flash, RAM, etc)
   - After that, it jumps to the ***entry point***, i.e. ***_start (x-heep/sw/device/lib/crt/crt0.S)***
3. Copy data & Vector Table Setup:
   - Copy data to the RAM if needed (based on runtime mode/boot procedure)
   - Sets the **vector table base address**…

The start-up Journey!

Line 120: x-heep/sw/device/lib/crt/crt0.S.tpl

3. Vector Table Setup:
   - The processor sets the **vector table base address…**

```
la   a0, __vector_start
ori  a0, a0, 0x1
csrw mtvec, a0
```

Instruction	Meaning
la a0, __vector_start	Loads the base address of the **interrupt vector table** (labelled __vector_start). This is defined in **the linker script**.
ori a0, a0, 0x1	Sets the **LSB to 1** to enable **vectored mode**. This means: when an interrupt occurs, the CPU will jump to an offset from mtvec **based on the interrupt ID**.
csrw mtvec, a0	Writes the modified address into the **mtvec CSR** (**Machine Trap Vector Register**), telling the CPU where to start looking for ISRs.

The start-up Journey!

Line 33: x-heep/sw/device/lib/crt/crt0.S.tpl

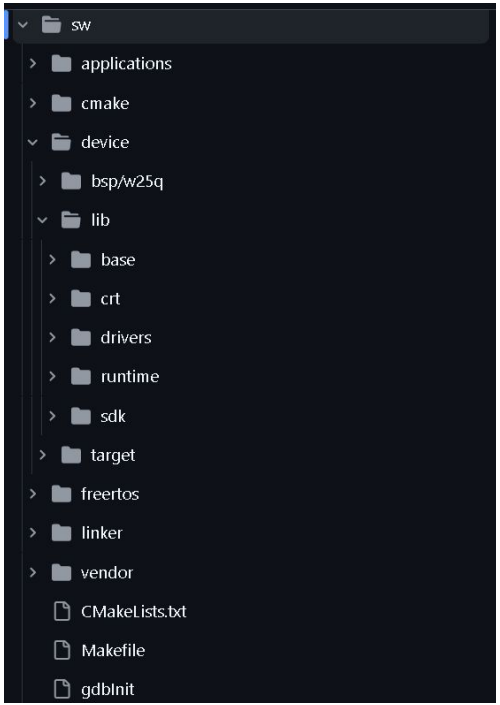
```
32      /* initialize stack pointer */
33        la sp, _sp
```

Line 290: x-heep/sw/linker/linker.ld.tpl

```
290         PROVIDE( _sp = .);
```

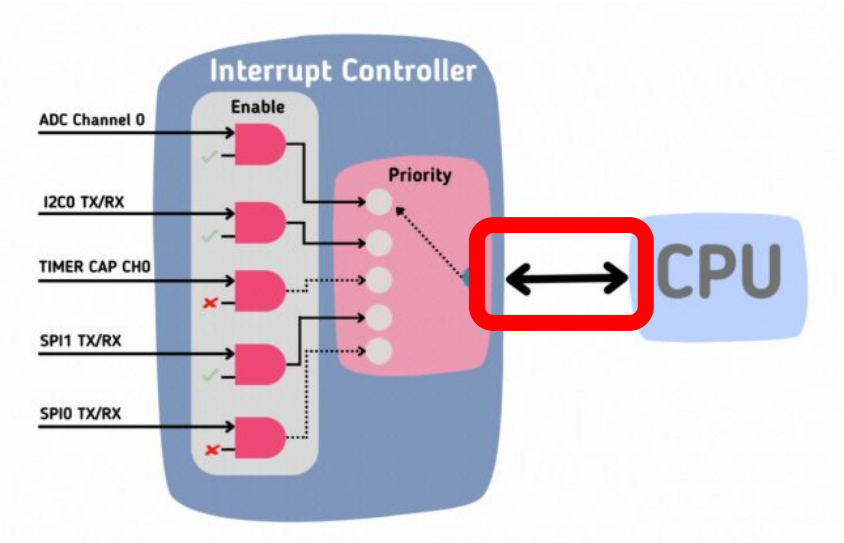
4. Stack pointer initialization (sp / can be done before/after vector table)
   - Initial SP value is loaded from a reserved RAM section
   - Ensures valid stack operations during runtime
5. (if not done before) Copy/Relocate Sections
   - Copy .data section from LMA to VMA
   - Zero-out .bss section
6. Magic: __libc_init_array()
   - e.g.: Ensure all global objects are ready before *main()* runs
7. *Call main() !*

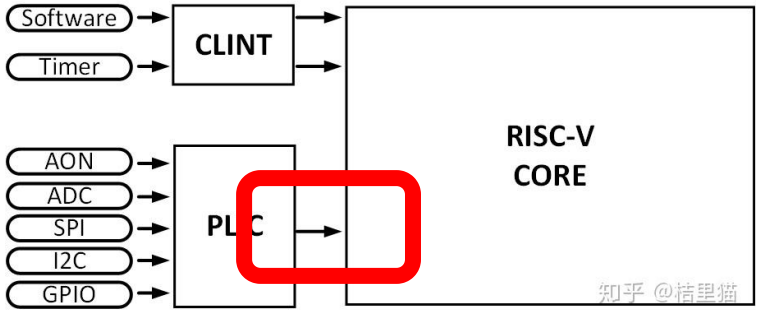
General xheep/sw structure



Item	Description
applications	Software examples and demos
cmake	Build system configuration files
device	Platform/device-specific support
bsp/w25q	Support for the W25Q flash device
lib	Shared software libraries
base	Basic utilities and core helpers
crt	C runtime startup code
drivers	Peripheral drivers
runtime	Execution-time support code
sdk	Development support utilities
target	Target-specific configuration
freertos	RTOS-related sources and integration
linker	Linker scripts and memory mapping
vendor	External imported components
CMakeLists.txt	Main CMake build description
Makefile	Main make-based build entry
gdbinit	Debugger startup configuration

Interrupts Unplugged: PLIC





Interrupts Unplugged: PLIC

x-heep / sw / device / lib / crt / vecto

Code Blame 132 lines (127 loc) ·

```
40      //  9 : supervisor extern
41      j __no_irq_handler
42      // 10 : unmapped
43      j __no_irq_handler
44      // 11 : machine external
45      j handler_irq_external
```

x-heep / hw / core-v-mini-mcu / peripheral_subsystem.sv.tpl

Code Blame 664 lines (595 loc) · 21.5 KB Raw

```
161
162     // Assign internal interrupts
163     assign intr_vector[${interrupts["null_intr"]}] = 1'b0;  // ID [0] is a special case and must be t
164     assign intr_vector[${interrupts["uart_intr_tx_watermark"]}] = uart_intr_tx_watermark;
165     assign intr_vector[${interrupts["uart_intr_rx_watermark"]}] = uart_intr_rx_watermark;
166     assign intr_vector[${interrupts["uart_intr_tx_empty"]}] = uart_intr_tx_empty;
167     assign intr_vector[${interrupts["uart_intr_rx_overflow"]}] = uart_intr_rx_overflow;
168     assign intr_vector[${interrupts["uart_intr_rx_frame_err"]}] = uart_intr_rx_frame_err;
169     assign intr_vector[${interrupts["uart_intr_rx_break_err"]}] = uart_intr_rx_break_err;
170     assign intr_vector[${interrupts["uart_intr_rx_timeout"]}] = uart_intr_rx_timeout;
171     assign intr_vector[${interrupts["uart_intr_rx_parity_err"]}] = uart_intr_rx_parity_err;
172     assign intr_vector[${interrupts["gpio_intr_31"]}:${interrupts["gpio_intr_8"]}] = gpio_intr;
173     assign intr_vector[${interrupts["intr_fmt_watermark"]}] = i2c_intr_fmt_watermark;
174     assign intr_vector[${interrupts["intr_rx_watermark"]}] = i2c_intr_rx_watermark;
175     assign intr_vector[${interrupts["intr_fmt_overflow"]}] = i2c_intr_fmt_overflow;
```

Interrupts Unplugged: PLIC

x-heep / sw / device / lib / drivers / rv_plic / rv_plic.c

Code Blame 447 lines (359 loc) · 13.1 KB

```c

void plic_reset_handlers_list(void)
{
  handlers[NULL_INTR] = &handler_irq_dummy;

  for( uint8_t i = NULL_INTR +1; i < QTY_INTR; i++ )
  {
    if ( i <= UART_ID_END)
    {
      handlers[i] = &handler_irq_uart;
    }
    else if ( i <= GPIO_ID_END)
    {
      handlers[i] = &handler_irq_gpio;
    }
    else if ( i <= I2C_ID_END)
    {
      handlers[i] = &handler_irq_i2c;
    }
```

x-heep / sw / device / lib / drivers / rv_plic / rv_plic.c

Code Blame 447 lines (359 loc) · 13.1 KB

```c

void handler_irq_external(void)
{
  uint32_t int_id = NULL_INTR;
  plic_result_t res = plic_irq_claim(&int_id);

  // Calls the proper handler
  handlers[int_id](int_id);
  plic_irq_complete(&int_id);
}
```

<driver>_structs.h generation

x-heep / hw / vendor / lowrisc_opentitan / hw / ip / rv_timer / data / rv_timer.hjson

Code Blame 215 lines (214 loc) · 5.44 KB

```
34      no_auto_intr_regs: "true",
35      regwidth: "32",
36      registers: [
37        { multireg: {
38            name: "CTRL",
39            desc: "Control register",
40            count: "N_HARTS",
41            cname: "TIMER",
42            swaccess: "rw",
43            hwaccess: "hro",
44            fields: [
45              { bits: "0", name: "active", desc: "If 1, time
46            ],
47          }
48        },
49        { skipto: "0x100" },
50        { name: "CFG0",
51          desc: "Configuration for Hart 0",
52          swaccess: "rw",
53          hwaccess: "hro",
54          fields: [
55            { bits: "11:0", name: "prescale", desc: "Prescaler to generate tick" },
56            { bits: "23:16", name: "step", resval: "0x1", desc: "Incremental value for each tick" },
57          ],
58        },
```

x-heep / util / periph_st

Name

..

periph_structs.tpl

periph_structs_gen.py

x-heep / sw / device / lib / drivers / rv_timer / rv_timer_structs.h

Code Blame 130 lines (88 loc) · 5.59 KB

```
40    /*****************************************************************************/
41    /**                                                                         **/
42    /**                        DEFINITIONS AND MACROS                           **/
43    /**                                                                         **/
44    /*****************************************************************************/
45
46    #define rv_timer_peri ((volatile rv_timer *) RV_TIMER_START_ADDRESS)
47
48    /*****************************************************************************/
49    /**                                                                         **/
50    /**                        TYPEDEFS AND STRUCTURES                          **/
51    /**                                                                         **/
52    /*****************************************************************************/
53
54
55
56    typedef struct {
57
58      uint32_t CTRL0;                          /*!< Control register*/
59
60      uint32_t _reserved_0[63];                /*!< reserved addresses*/
61
62      uint32_t CFG0;                           /*!< Configuration for Hart 0
```

Peeking under the hood: OpenOCD & GDB

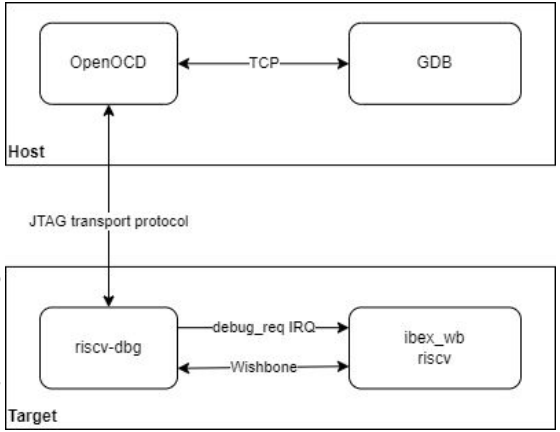
GDB — The Debugger GDB knows how to:

- Load binaries (ELF), set breakpoints, single-step, inspect memory/registers
- Manage C-level symbolic debug info

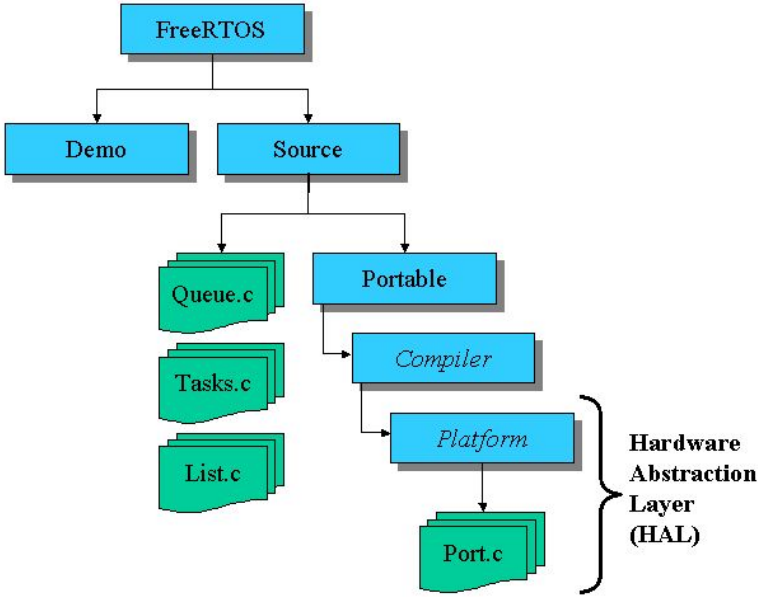
- **But GDB assumes it's talking to a running CPU via a protocol** (JTAG)
- It **does not know how to talk directly to real hardware** or JTAG lines on its own.

OpenOCD — The Hardware Debug Bridge OpenOCD is the **translator between GDB and the silicon**.

- It speaks low-level **JTAG protocol** to control the CPU.
- It exposes a **GDB remote target server** on TCP (localhost:3333).



Spelunking into the RTOS depth: FreeRTOS & X-HEEP



x-heep / sw / CMakeLists.txt

Code Blame 567 lines (481 loc) · 22.4 KB

```
# # # #
```

THANK YOU!

jose.miranda@upm.es